SESSION 2: PACKAGES

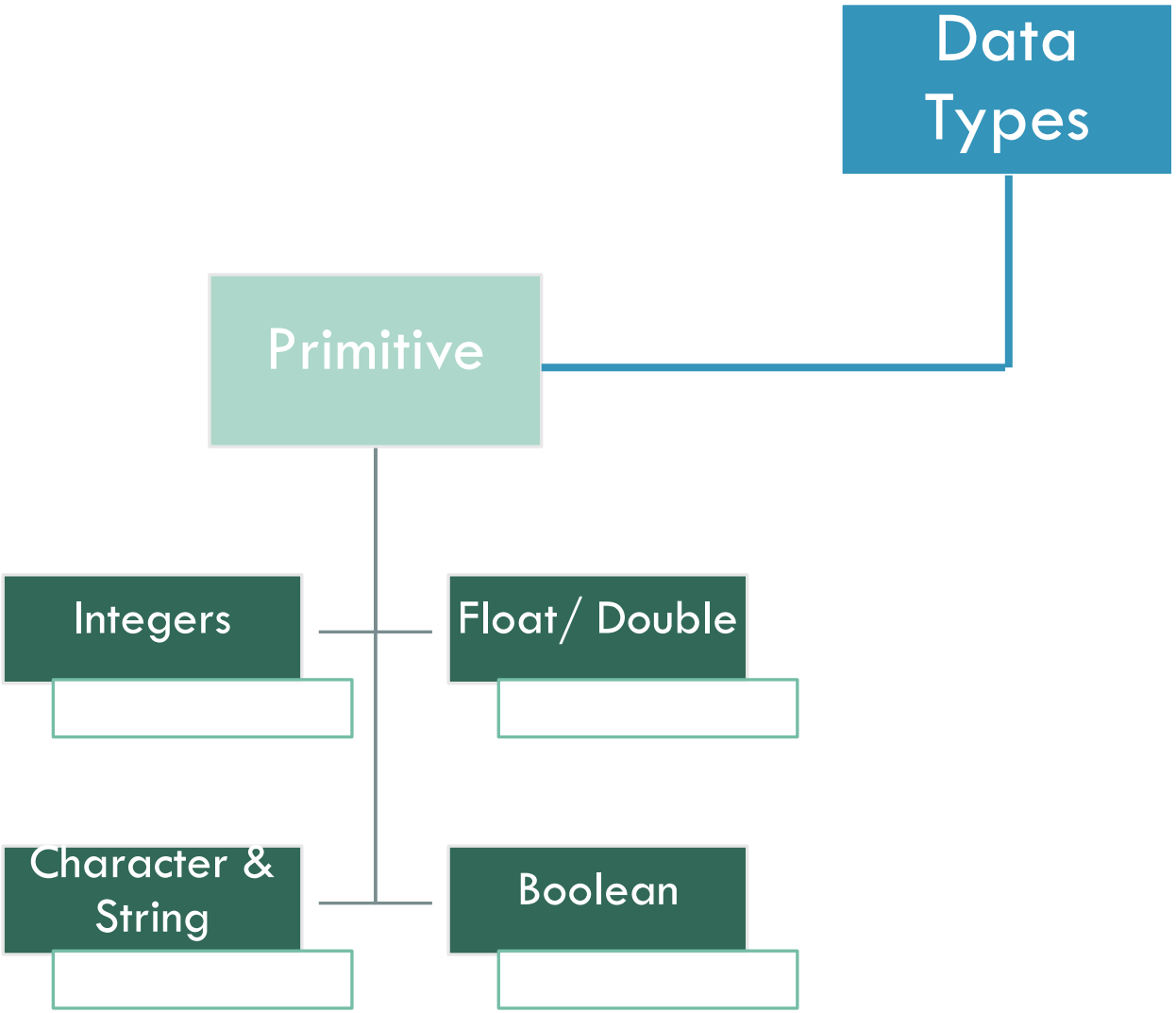
DR. SOFIA GIL-CLAVEL

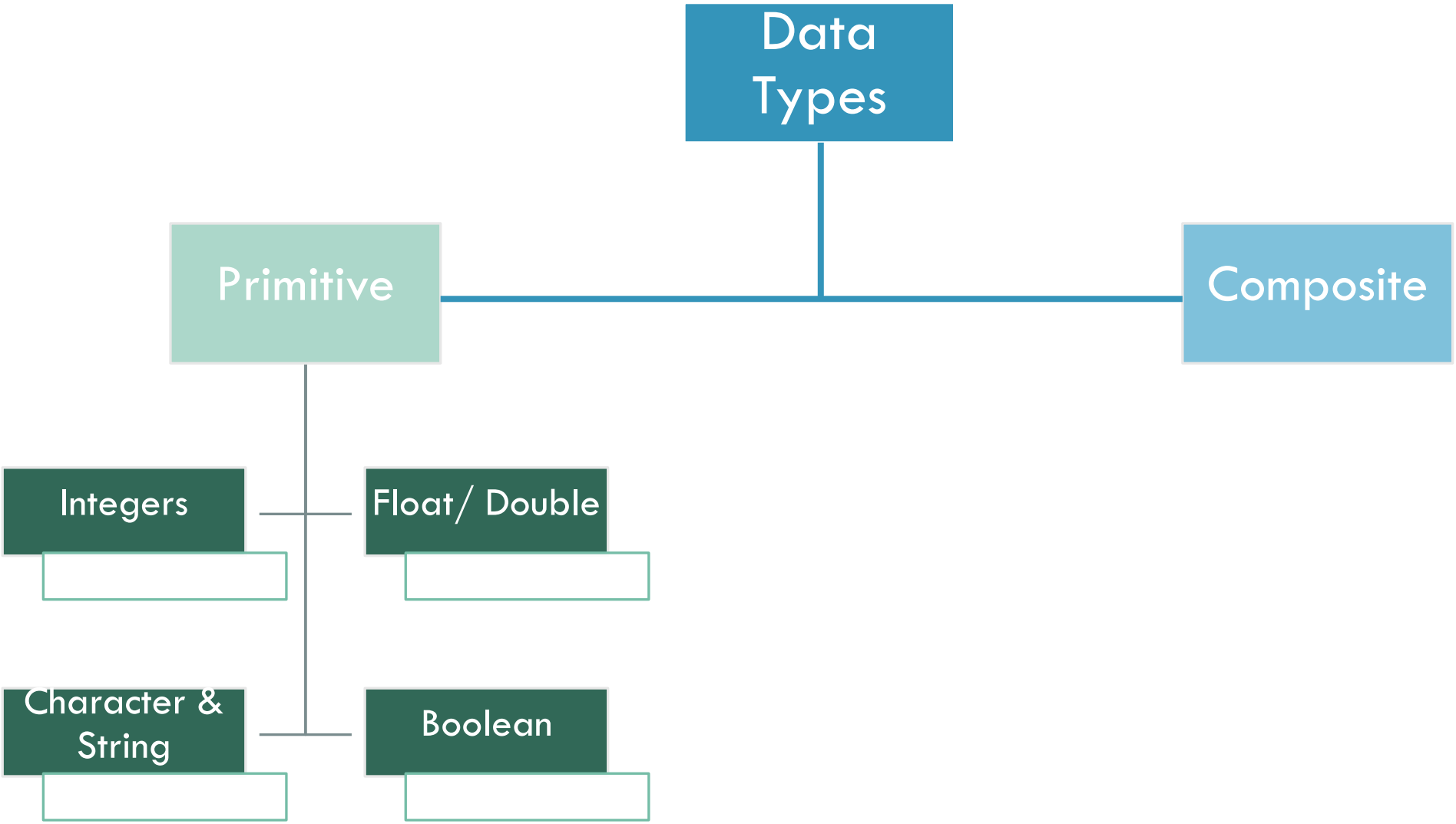
- Session 1: Recap
- Packages
- Tidyverse
- Learning to learn

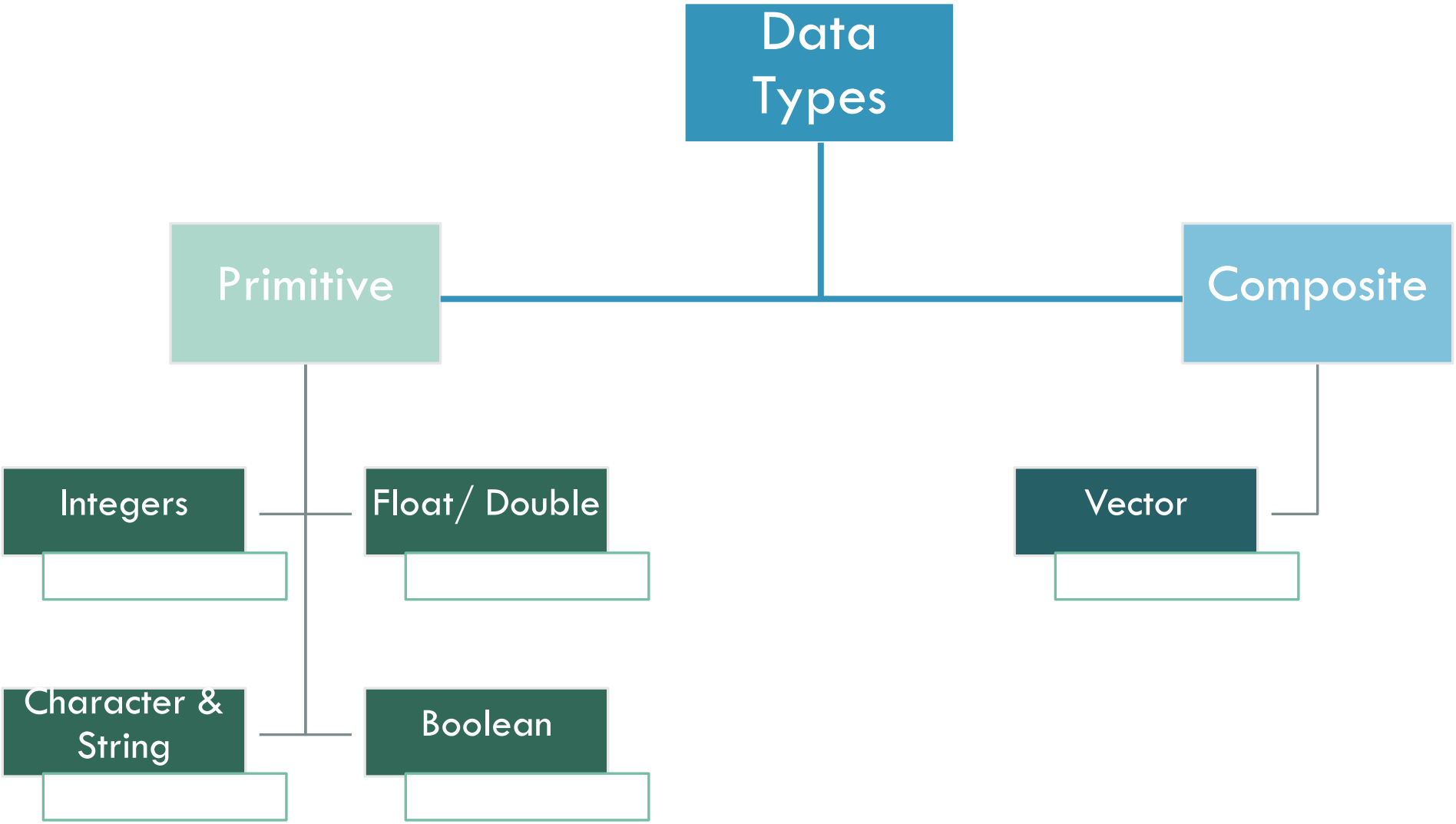
1. SESSION 1: RECAP

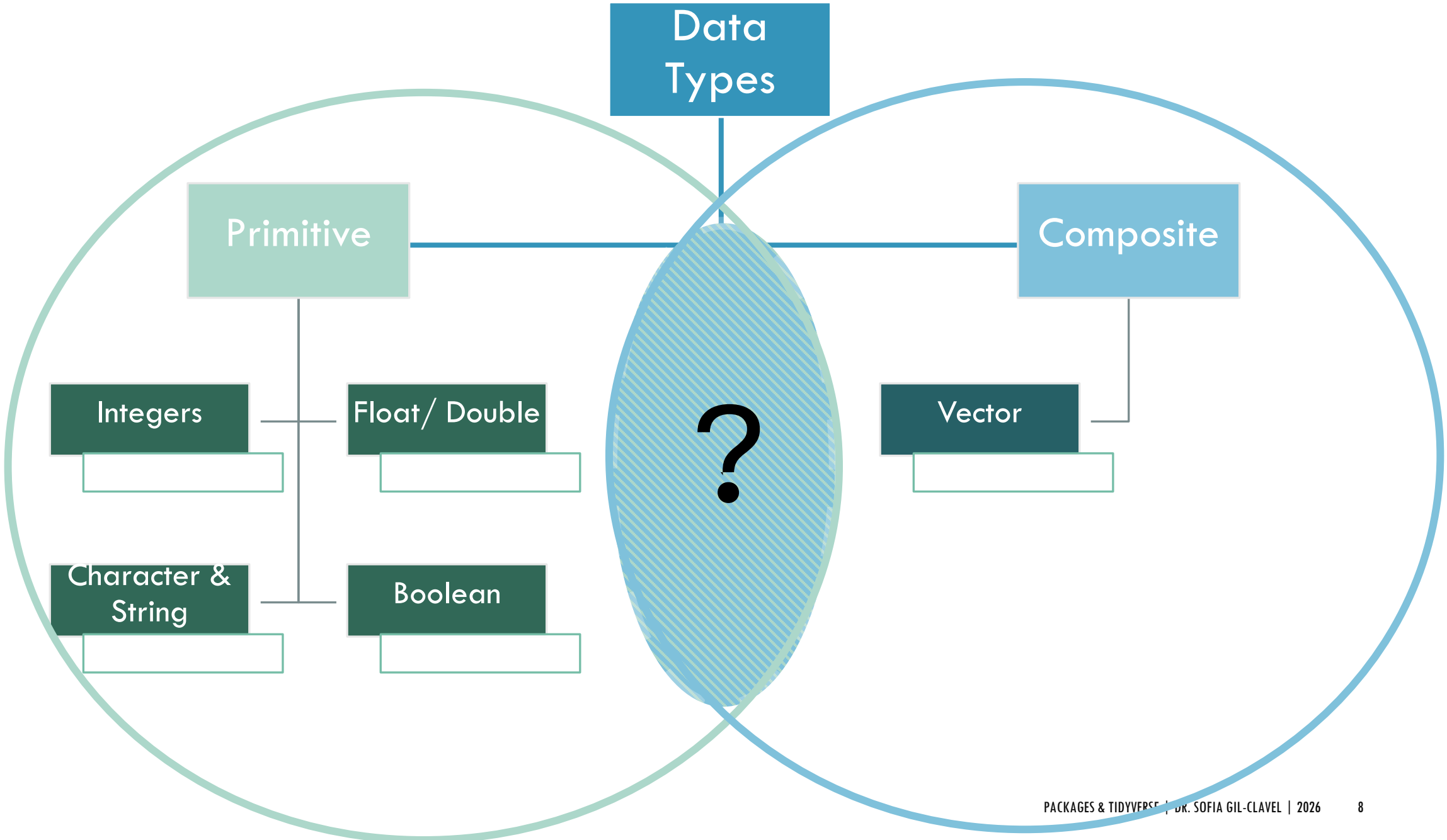
Data Types

Data Types
Primitive


Data Types
Primitive
Integers
Float/ Double
Character & String
Boolean


Data Types
Primitive
Composite
Integers
Float/ Double
Character & String
Boolean


Data Types
Primitive
Composite
Integers
Float/ Double
Character & String
Boolean
Vector


Data Types
Primitive
Composite
Integers
Float/ Double
Character & String
Boolean
?
Vector

Data Types

Primitive

Composite

Integers

Float/ Doubl

Character & String

Boolean

Vector

Data Frame

```
> n = c(2, 3, 5)
> s = c("aa", "bb", "cc")
> b = c(TRUE, FALSE, TRUE)
> df = data.frame(n, s, b)
```

PRIMITIVE DATA AND ITS OPERATORS

Operators					
Arithmetic		**Comparative**		**Logic**	
+	addition	<	less than	! x	logic NO
-	subtraction	>	more than	x & y	element-wise AND
*	multiplication	<=	less or equal than	x && y	single comparison AND
/	division	>=	more or equal than	x | y	element-wise OR
^	power	==	equal than	x || y	single comparison OR
%/%	integer division	!=	different than	xor(x,y)	exclusive OR

OPEN AND SAVE

Open: read.csv()

You can open “csv” files using this function.

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",
         dec = ".", fill = TRUE, comment.char = "", ...)
```

Save: write.csv()

```
write.table(x, file = "", append = FALSE, quote = TRUE, sep = " ",
            eol = "\n", na = "NA", dec = ".", row.names = TRUE,
            col.names = TRUE, qmethod = c("escape", "double"),
            fileEncoding = "")

write.csv(...)
```

EXERCISE 1.1

Open the "df" dataframe you saved last time:

1. Create a variable to store the path to the folder where you saved the dataframe.

```
DIR="Folder location"
```

2. Use the function "paste0()" to concatenate the variable and the name of the file. The file must include the extension, e.g. ".txt" or ".csv".

```
FILE=paste0(DIR, "My1stDataframe.csv")
```

3. Use the function "read.table()" or "read.csv()" to open your dataframe in R.

   Hint: Do not forget to assign the dataframe to a variable.

```
df <- read.csv(FILE)
```

DATAFRAME FUNCTIONS

The dataframe name

The columns:
names(mtcars)
or
colnames(mtcars)

```
> mtcars
                   mpg cyl disp  hp drat   wt ...
Mazda RX4         21.0   6  160 110 3.90 2.62 ...
Mazda RX4 Wag     21.0   6  160 110 3.90 2.88 ...
Datsun 710        22.8   4  108  93 3.85 2.32 ...
                  ...........
```

The row names:
row.names(mtcars)

A variable:
mtcars$disp

A value or element:
mtcars$wt[3]

EXERCISE 1.2

Create a copy of the “df” dataframe called “df2”.

Hint: Use the assign operator “=” or “<-”

To “df2” apply the functions from the previous slide together with the operator to do the following:

- Obtain the names of your dataframe’s columns.
- Obtain the result of adding up the values stored in the column “n”.

  Hint: Use the functions “$” and “[]” together with the operator “+”.
- Create a new column named “b2” with the result of obtaining the opposite of the values in “b”.

  Hint: Use the function “$” to create the new column and use the operator “!” to invert the values in “b”.

SOME USEFUL FUNCTIONS

`sum(x)`
`prod(x)`
`max(x)`
`min(x)`
`which.max(x)`
`which.min(x)`
`range(x)`
`length(x)`
`mean(x)`
`median(x)`
`var(x)` o `cov(x)`
`cor(x)`
`var(x, y)` o `cov(x, y)`
`cor(x, y)`

`round(x, n)`
`rev(x)`
`sort(x)`
`rank(x)`
`log(x, base)`
`scale(x)`
`pmin(x,y,...)`
`pmax(x,y,...)`
`cumsum(x)`
`cumprod(x)`
`cummin(x)`
`cummax(x)`

EXERCISE 1.3

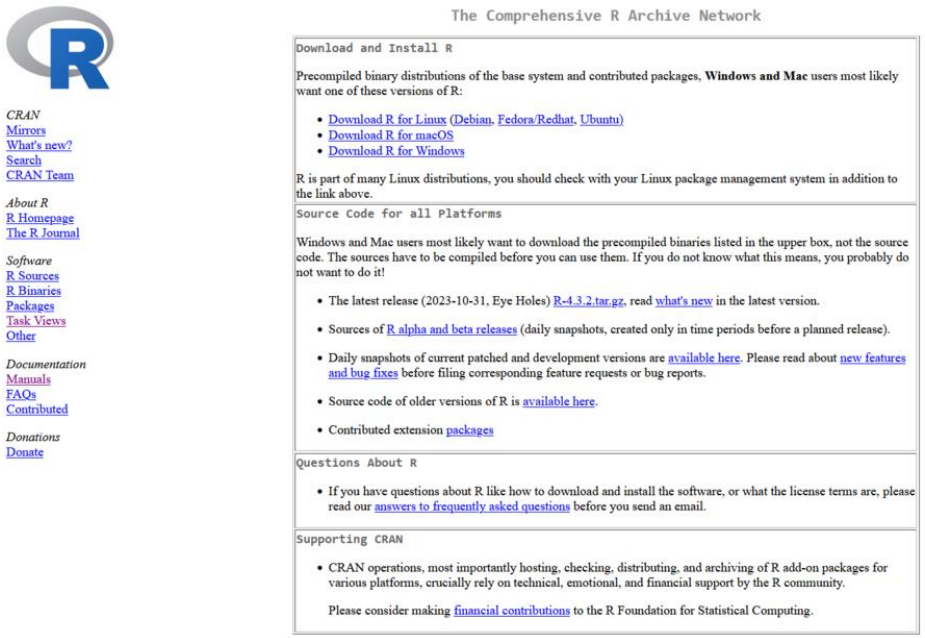
Using the functions from the previous slide obtain the following values:

- Obtain the result of adding up the values stored in the column “n”.
- Obtain the correlation between “n” and “n2”.

2. PACKAGES

WHAT IS A PACKAGE?

- R packages are extensions to the R statistical programming language.
- R packages contain code, data, and documentation in a standardized collection format that can be installed by users of R, typically via a centralized software repository such as CRAN (the Comprehensive R Archive Network).



https://cran.r-project.org/

LET'S USE OUR FIRST PACKAGE

We will download, open, and use the package "rio". For this, we will use the following functions:

- install.packages("rio")

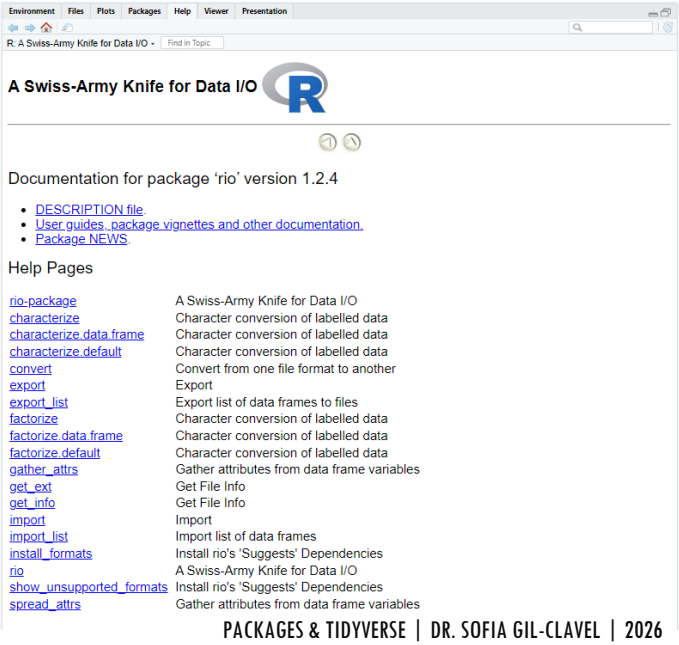
  This function downloads the package "rio" (which must be written between quotes) and installs it in R.

- library("rio")

  This function opens the package "rio", so that we can use it in R.

If you want to learn more about a package, R offers a couple of ways to do so.

1. Go to the "Packages" panel.
2. Look for the package "rio" and click on it.
3. This will open a file with the links to some tutorials, and the links to all the functions and their help pages.



EXERCISE 2.1

Use the following functions to save and open the “df2” dataframe:

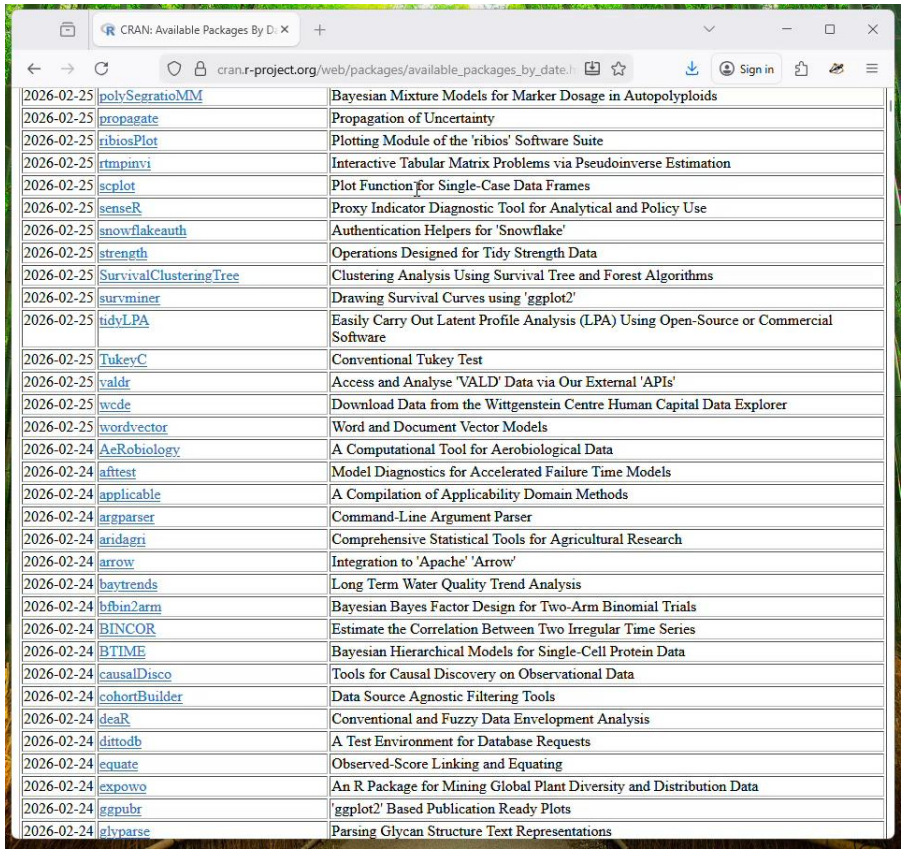
- export
- import

Hint: In the “rio” package information, look for those functions. Then read the functions documentation and follow the examples at the end.

HOW MANY PACKAGES ARE OUT THERE?

- There are thousands of packages!
- This makes it hard to keep track of the best ones or the most reliable ones!

Therefore, we need to be careful when using R-packages. As plenty of them, may not be the best for our goal.



2026-02-25	polySegratioMM	Bayesian Mixture Models for Marker Dosage in Autopolyploids
2026-02-25	propagate	Propagation of Uncertainty
2026-02-25	ribiosPlot	Plotting Module of the 'ribios' Software Suite
2026-02-25	rtmpinvi	Interactive Tabular Matrix Problems via Pseudoinverse Estimation
2026-02-25	scplot	Plot Function for Single-Case Data Frames
2026-02-25	senseR	Proxy Indicator Diagnostic Tool for Analytical and Policy Use
2026-02-25	snowflakeauth	Authentication Helpers for 'Snowflake'
2026-02-25	strength	Operations Designed for Tidy Strength Data
2026-02-25	SurvivalClusteringTree	Clustering Analysis Using Survival Tree and Forest Algorithms
2026-02-25	survminer	Drawing Survival Curves using 'ggplot2'
2026-02-25	tidyLPA	Easily Carry Out Latent Profile Analysis (LPA) Using Open-Source or Commercial Software
2026-02-25	TukeyC	Conventional Tukey Test
2026-02-25	valdr	Access and Analyse 'VALD' Data via Our External 'APIs'
2026-02-25	wcde	Download Data from the Wittgenstein Centre Human Capital Data Explorer
2026-02-25	wordvector	Word and Document Vector Models
2026-02-24	AeRobiology	A Computational Tool for Aerobiological Data
2026-02-24	afttest	Model Diagnostics for Accelerated Failure Time Models
2026-02-24	applicable	A Compilation of Applicability Domain Methods
2026-02-24	argparser	Command-Line Argument Parser
2026-02-24	aridagri	Comprehensive Statistical Tools for Agricultural Research
2026-02-24	arrow	Integration to 'Apache' 'Arrow'
2026-02-24	baytrends	Long Term Water Quality Trend Analysis
2026-02-24	bfbin2arm	Bayesian Bayes Factor Design for Two-Arm Binomial Trials
2026-02-24	BINCOR	Estimate the Correlation Between Two Irregular Time Series
2026-02-24	BTIME	Bayesian Hierarchical Models for Single-Cell Protein Data
2026-02-24	causalDisco	Tools for Causal Discovery on Observational Data
2026-02-24	cohortBuilder	Data Source Agnostic Filtering Tools
2026-02-24	deaR	Conventional and Fuzzy Data Envelopment Analysis
2026-02-24	dittodb	A Test Environment for Database Requests
2026-02-24	equate	Observed-Score Linking and Equating
2026-02-24	expowo	An R Package for Mining Global Plant Diversity and Distribution Data
2026-02-24	ggpubr	'ggplot2' Based Publication Ready Plots
2026-02-24	glyparse	Parsing Glycan Structure Text Representations

https://cran.r-project.org/web/packages/available_packages_by_date.html

3. TIDYVERSE

❖ TIDYVERSE: HTTPS://WWW.TIDYVERSE.ORG/

R packages for data science

The tidyverse is an opinionated **collection of R packages** designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

EXERCISE 3.1 + 10 MINS BREAK

Install the package “tidyverse”.

Hint: Use the function install.packages()

Open the package “tidyverse” in R.

Hint: Use the function library()

THE DATA

From now onwards, we will use the data "iris".

This dataframe is part of the Base R package, so you just need to type its name in the console to see it.

```
Console   Terminal   Background Jobs
R 4.4.2 · ~/
>
> iris
    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1            5.1         3.5          1.4         0.2     setosa
2            4.9         3.0          1.4         0.2     setosa
3            4.7         3.2          1.3         0.2     setosa
4            4.6         3.1          1.5         0.2     setosa
5            5.0         3.6          1.4         0.2     setosa
6            5.4         3.9          1.7         0.4     setosa
7            4.6         3.4          1.4         0.3     setosa
8            5.0         3.4          1.5         0.2     setosa
9            4.4         2.9          1.4         0.2     setosa
10           4.9         3.1          1.5         0.1     setosa
```

Let's save it in the variable "IRIS" to play around with it.

LET'S FOCUS ON DPLYR

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges.

https://dplyr.tidyverse.org/

WE WILL START WITH THE FOLLOWING FUNCTIONS:

https://dplyr.tidyverse.org/

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

These all combine naturally with `group_by()` which allows you to perform any operation "by group". You can learn more about them in `vignette("dplyr")`. As well as these single-table verbs, dplyr also provides a variety of two-table verbs, which you can learn about in `vignette("two-table")`.

If you are new to dplyr, the best place to start is the data transformation chapter in R for Data Science.

MUTATE: CREATE, MODIFY, AND DELETE COLUMNS

mutate() creates new columns that are functions of existing variables. It can also modify (if the name is the same as an existing column) and delete columns (by setting their value to NULL).

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- mutate(IRIS, …)
```

HOW DOES MUTATE WORK?

dplyr will apply a function to the elements of the column that you choose.

Let's say that you want to add 2 to the column "Sepal.Width".

Sepal.Width
3.5
3.0
3.2
3.1
3.6
3.9
3.4
3.4
2.9
3.1

Then you do:

mutate(IRIS, Sepal.Width2= Sepal.Width+2)

1. Creates the new column

2. Then it stores in each element the result of performing the operation.

Sepal.Width2	Sepal.Width	
5.5	3.5	+2
5.0	3.0	+2
5.2	3.2	+2
5.1	3.1	+2
5.6	3.6	+2
5.9	3.9	+2
5.4	3.4	+2
5.4	3.4	+2
4.9	2.9	+2
5.1	3.1	+2

MUTATE: CREATE, MODIFY, AND DELETE COLUMNS

mutate() creates new columns that are functions of existing variables. It can also modify (if the name is the same as an existing column) and delete columns (by setting their value to NULL).

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- mutate(IRIS, …)
```

Exercise 3.2

1. Create a column called "Petal.Width2" that contains the result of multiplying the column "Petal.Width" by two.
2. Modify the column "Sepal.Length" by dividing its elements by its variance.

   Hint: Use the function var().
3. Delete the column "Species".

SELECT: KEEP OR DROP COLUMNS USING THEIR NAMES AND TYPES

select(): Select (and optionally rename) variables in a data frame, using a concise mini-language that makes it easy to refer to variables based on their name (e.g. a:f selects all columns from a on the left to f on the right) or type (e.g. where(is.numeric) selects all numeric columns).

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- select(IRIS, …)
```

Exercise 3.3

1. Save the column "Sepal.Length" in the variable "SL".
2. Keep only the first 4 columns.

Hint: Use the function names() to check the names of the first four columns.

FILTER: KEEP OR DROP ROWS THAT MATCH A CONDITION

filter(): This function is used to subset a data frame, applying the expressions in ... to determine which rows should be kept.

In its core, this function creates a logical vector. So, if you want to keep the opposite, then you can use the operator "!".

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- filter(IRIS, …)
```

HOW DOES FILTER WORK?

Let's say that you want to keep only the elements in "Species" that are "setosa".

```
Species
<fct>
setosa
setosa
setosa
versicolor
versicolor
versicolor
virginica
virginica
virginica
```

Then you do:
filter(IRIS, Species == "setosa")

1. Performs the comparison by column element.

```
Species
<fct>
setosa      == "setosa" → TRUE
setosa      == "setosa" → TRUE
setosa      == "setosa" → TRUE
versicolor  == "setosa" → FALSE
versicolor  == "setosa" → FALSE
versicolor  == "setosa" → FALSE
virginica   == "setosa" → FALSE
virginica   == "setosa" → FALSE
virginica   == "setosa" → FALSE
```

2. Then it keeps only those rows where the comparison was "TRUE".

```
# A tibble: 50 × 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <fct>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
10          4.9         3.1          1.5         0.1 setosa
# ℹ 40 more rows
# ℹ Use `print(n = ...)` to see more rows
```

FILTER: KEEP OR DROP ROWS THAT MATCH A CONDITION

filter(): This function is used to subset a data frame, applying the expressions in ... to determine which rows should be kept.

In its core, this function creates a logical vector. So, if you want to keep the opposite, then you can use the operator "!".

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

IRIS <- filter(IRIS, …)

Exercise 3.4

1. In a new variable, save the resulting dataframe after keeping only those values that are bigger than "Sepal.Width" mean.

   Hint: Use the function mean() and the operator ">".

2. In a new variable, save the resulting dataframe after keeping only those values that are equal or smaller than "Sepal.Width" mean.

   Hint: Use the operator "!".

SUMMARIS(Z)E: SUMMARIZE EACH GROUP DOWN TO ONE ROW

summarise() or summarize(): creates a new data frame. It returns one row for each combination of grouping variables; if there are no grouping variables, the output will have a single row summarizing all observations in the input. It will contain one column for each grouping variable and one column for each of the summary statistics that you have specified.

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- summarize(IRIS, …)
```

HOW DOES SUMMARIZE WORK?

Let's say that you want to get the minimum of the columns "Sepal.Length" and Sepal.Width"

```
Sepal.Length Sepal.Width
       <dbl>       <dbl>
         5.1         3.5
         4.9         3
         4.7         3.2
         4.6         3.1
         5           3.6
         5.4         3.9
         4.6         3.4
         5           3.4
         4.4         2.9
         4.9         3.1
```

Then you do:

summarise(IRIS, min(Sepal.Length),min(Sepal.Width))

1. Performs the function on each column by passing the column as a vector into the function.

2. Then it returns the output in dataframe format.

```
min(Sepal.Length) min(Sepal.Width)
              4.3                 2
```

SUMMARIS(Z)E: SUMMARIZE EACH GROUP DOWN TO ONE ROW

summarise() or summarize(): creates a new data frame. It returns one row for each combination of grouping variables; if there are no grouping variables, the output will have a single row summarizing all observations in the input. It will contain one column for each grouping variable and one column for each of the summary statistics that you have specified.

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

```
IRIS <- summarize(IRIS, …)
```

Exercise 3.5:

For the following, do not save the modified dataframe.

- Obtain the minimum value of each of the numerical columns.

  Hint: Use the function min().

ARRANGE: ORDER ROWS USING COLUMN VALUES

arrange(): orders the rows of a data frame by the values of selected columns.

WARNING:

For any function to have an effect on your dataframe, you need to save the modified variable in itself:

IRIS <- arrange(IRIS, …)

Exercise 3.6:

Perform the following inside the function "View()" to visualize the effect on your dataframe.

1. Order the dataframe by the "Petal.Width" values in ascending order.
2. Order the dataframe by "Petal.Length" values in descending order.

NOW, LET'S INTRODUCE THE PIPE FUNCTION

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

Here, "equivalent" is not technically exact: evaluation is non-standard, and the left-hand side is evaluated before passed on to the right-hand side expression. However, in most cases this has no practical implication.

https://magrittr.tidyverse.org/reference/pipe.html

SOME PIPED EXAMPLES FROM THE PREVIOUS EXERCISES

Create a column called “Petal.Width2” that contains the result of multiplying the column “Petal.Width” by two.

What we did:

```
mutate(IRIS, Petal.Width2 = 2 * Petal.Width)
```

Using pipes:

```
IRIS %>% mutate(Petal.Width2 = 2 * Petal.Width)
```

SOME PIPED EXAMPLES FROM THE PREVIOUS EXERCISES

Perform the following inside the function "View()" to visualize the effect on your dataframe.

1. Order the dataframe by the "Petal.Width" values in ascending order.

What we did:

```
View(arrange(IRIS, Petal.Width))
```

Using pipes:

```
IRIS %>% arrange(Petal.Width) %>% View()
```

EXERCISE 3.7

Using the function "%>%" perform the following data transformations to the original "iris" dataframe. Save the result in the variable "IRIS_PIPE".

1. Create the column "LOGICAL" indicating whether "Sepal.Length" is bigger than its mean.

   Hint: Use the functions mutate() and mean(), together with the operator ">".

2. Keep only the rows where "Species" is different from "setosa".

   Hint: Use the functions select() together with the operator "!=".

3. Order the dataframe by the "LOGICAL" column.

   Hint: Use the function arrange().

Group by one or more variables

Source: R/group-by.R

Most data operations are done on groups defined by variables. `group_by()` takes an existing tbl and converts it into a grouped tbl where operations are performed "by group". `ungroup()` removes grouping.

```
group_by(.data, ..., .add = FALSE, .drop = group_by_drop_default(.data))

ungroup(x, ...)
```

Source: https://dplyr.tidyverse.org/reference/group_by.html

HOW DOES GROUP_BY WORK?

Let's say that you want to get the minimum of the columns "Sepal.Length" and Sepal.Width" by Species.

Sepal.Length	Sepal.Width	Species
<dbl>	<dbl>	<fct>
5.2	3.4	setosa
5.5	4.2	setosa
4.8	3.4	setosa
6.6	3	versicolor
6.6	2.9	versicolor
5.8	2.7	versicolor
5.8	2.7	virginica
6.7	3.3	virginica
7.4	2.8	virginica

Then you do:

```
iris%>%
group_by(Species)%>%
summarise(min(Sepal.Length),min(Sepal.Width))
```

1. group_by(Species) tells R that you are going to perform the following operations by Species.

2. summarise performs the function on each column by Species.

3. Then it returns the output in dataframe format.

Species	`min(Sepal.Length)`	`min(Sepal.Width)`
<fct>	<dbl>	<dbl>
setosa	4.3	2.3
versicolor	4.9	2
virginica	4.9	2.2

EXERCISE 3.7

Using the function "%>%" perform the following data transformations to the original "iris" dataframe. Save the results in different variables.

1. Create a new column with the maximum value of the column "Sepal.Width" by "Species".

   Hint: Use the functions group_by(),%>%, mutate(), and max().

2. Obtain the maximum value of the column "Sepal.Width" by "Species".

   Hint: Use the functions group_by(), %>%, summarize(), and max().

What is the difference between using mutate and summarize together with group_by?

4. LEARNING TO LEARN

NOW, LET'S INTRODUCE THE TIBBLES

https://tibble.tidyverse.org/

A **tibble**, or `tbl_df`, is a modern reimagining of the data.frame, keeping what time has proven to be effective, and throwing out what is not. Tibbles are data.frames that are lazy and surly: they do less (i.e. they don't change variable names or types, and don't do partial matching) and complain more (e.g. when a variable does not exist). This forces you to confront problems earlier, typically leading to cleaner, more expressive code. Tibbles also have an enhanced `print()` method which makes them easier to use with large datasets containing complex objects.

VIGNETTES

During this workshop, you have been using tibbles.
Any time you introduce a data.frame in a Dplyr function (except for few cases), it will coerce it into a tibble.
You can check this by typing in the console: "iris" and then any of the variables where we stored the result of using dplyr.

If you want to learn more about a package, R offers a couple of ways to do so.

1. Go to the "Packages" panel.

2. Look for the package "tibble" and click on it.

3. This will open a file with the links to some tutorials, and the links to all the functions and their help pages.

4. Click on "User guides, package vignettes and other documentation."

5. From there, press on "tibble::tibble".

Exercise 4.1:

Based on the tutorial:

- How do you create a tibble?
- Why is tibble better than data.frame?
- What are the main warnings when using a tibble?

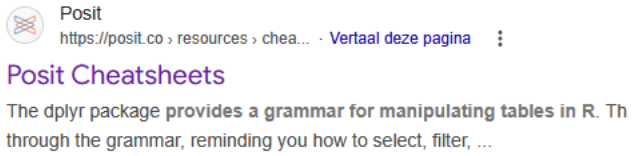
CHEATSHEETS

All the “tidyverse” packages come with a Cheatsheet.

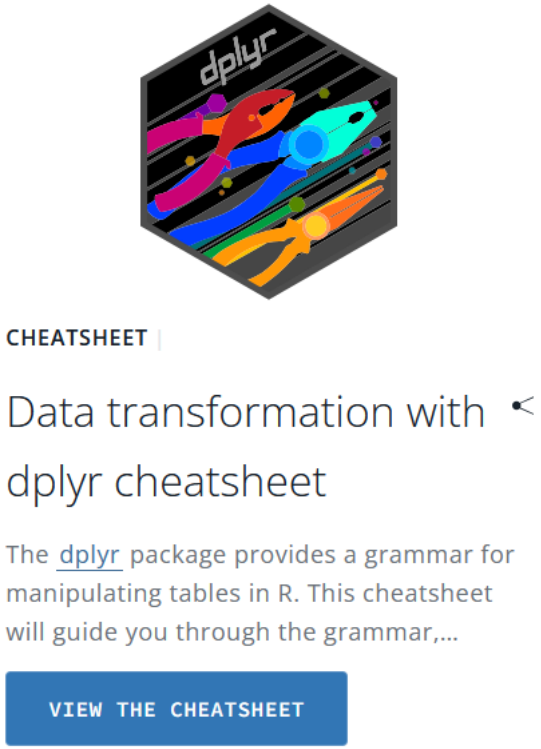
A Cheatsheet is an infographic with graphic explanations of what the package does and how the functions work.

1. Open your favorite browser (Google, Mozilla, Edge, etc.).
2. In the search section, type the words:

“Posit”, which is the name of company behind tidyverse together with “Cheatsheet”.



3. In there, you will see Cheatsheets for different packages.
4. Scroll down and find the one for “Dplyr”.



EXERCISE 4.2

1. Using tibble, create two dataframes with the following columns:

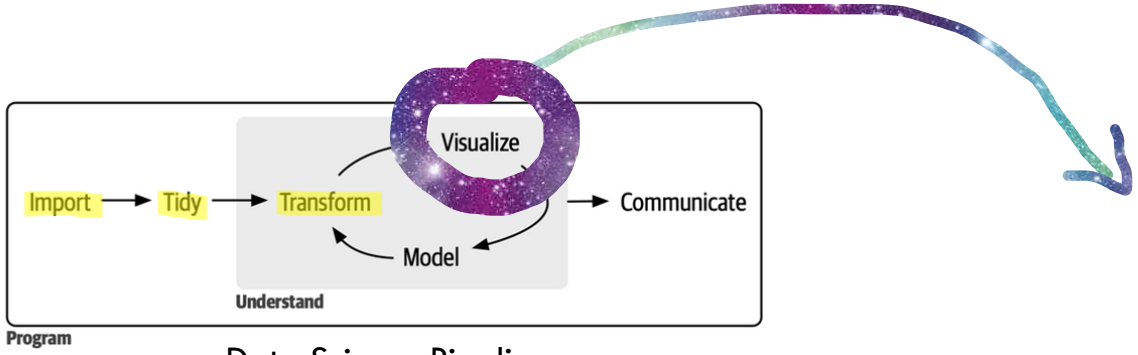
First dataframe	Second dataframe
COUNTRY<-c("MX", "DE", "NL", "MX", "NL")	COUNTRY<-c("MX", "DE", "NL")
YEAR<-c(2015, 2015, 2015, 2020, 2020)	YEAR<-c(2015, 2015, 2015, 2020, 2020, 2020)
GDP<-c(1.21, 3.3, 1.15, 1.12, 1.054)	POPULATION<-c(119.5, 82.2, 16.9, 137.2, 83.2, 17.4)

2. Follow the Dplyr Cheatsheet to join the dataframes by the column "COUNTRY" and "YEAR".

REFERENCES

- Albert, Jim, and Maria Rizzo. *R by Example: Concepts to Code*. Use R! New York, NY: Springer New York, 2012. https://doi.org/10.1007/978-1-4614-1365-3.
- Davies, Tilman M. *The Book of R: A First Course in Programming and Statistics*. San Francisco: No Starch Press, 2016. https://web.itu.edu.tr/~tokerem/The_Book_of_R.pdf
- Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. *R for data science*. " O'Reilly Media, Inc.", 2023. Accessed May 7, 2024. https://r4ds.hadley.nz/.

SESSION 3: GGPLOT2 AND THE GRAMMAR OF GRAPHS



Data Science Pipeline

ggplot2 is a system for declaratively creating graphics, based on The Grammar of Graphics. You provide the data, tell ggplot2 how to map variables to aesthetics, what graphical primitives to use, and it takes care of the details.

Source: https://ggplot2.tidyverse.org/

Join us!

https://forms.office.com/e/8Bgd2YsasJ

All about the lab:

https://societal-analytics.nl/

Contact us at:

analytics-lab.fsw@vu.nl.

https://sofiag1l.github.io/

THANKS!

Dr. Sofia Gil-Clavel